

JavaScript

Course Structure:

<u>Chapter</u>	<u>Topic</u>
1	Introduction to JavaScript
2	JavaScript Syntax, Variables & Data Types
3	Operators & Expressions
4	Conditional Statements
5	Loops
6	Functions
7	Arrays
8	Strings & String Methods
9	Objects
10	ES6+ Features
11	DOM
12	Events & Event Handling
13	Forms & Form Validation

Chapter

Topic

14

BOM

15

Error Handling

16

JavaScript Date & Math

17

Advanced Functions

18

Asynchronous JavaScript

19

Promises & Async/Await

20

Fetch API & JSON

21

Local Storage & Session Storage

22

Modules

23

OOP in JavaScript

24

JavaScript with HTML/CSS

25

Mini Projects

26

Final Project

CHAPTER 1 — INTRODUCTION TO JAVASCRIPT

1.1 What is JavaScript?

JavaScript is a high-level, interpreted programming language mainly used to make web pages interactive and dynamic.

HTML is used to create the **structure** of a webpage.

CSS is used to control the **appearance/design**.

JavaScript is used to add **behavior and functionality**.

Simple Example

Suppose we create a button:

```
<button>Click Me</button>
```

HTML creates the button.

CSS can make the button beautiful.

JavaScript can make something happen when the user clicks it.

For example:

```
alert("Hello MCA Students!");
```

When JavaScript executes, a message box appears.

1.2 HTML, CSS and JavaScript

A very important concept for students is:

HTML → Structure

CSS → Presentation

JavaScript → Behavior

Think of a house:

Web Technology

HTML

CSS

JavaScript

House

Structure of the house

Paint and decoration

Electricity, switches and functionality

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>JavaScript Demo</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Hello MCA Students</h1>
```

```
  <button onclick="alert ('Welcome to JavaScript!')">
```

```
    Click Me
```

```
  </button>
```

```
</body>
```

```
</html>
```

Here:

- <h1> → HTML
- <button> → HTML
- onclick → JavaScript event
- alert() → JavaScript function

1.3 History of JavaScript

JavaScript was created by **Brendan Eich** in **1995** while working at **Netscape**.

Initially, it was developed for adding scripting capabilities to web browsers.

It was originally called **Mocha**, then **LiveScript**, and eventually **JavaScript**.



Brendan Eich (born July 4, 1961) *is an American computer programmer and technology executive. He created the JavaScript programming language.*

Important point

JavaScript and Java are **different programming languages**.

The names are similar, but they have different:

- Syntax
- Execution models
- Type systems
- Runtime environments
- Applications

1.4 Why is JavaScript called JavaScript?

JavaScript was named JavaScript largely for marketing/branding reasons during its early development. It is not a derivative of Java.

Java vs JavaScript

Java

General-purpose programming language

Strong/static type system

Runs on JVM

.java source files

Class-based OOP

Common in enterprise/backend/Android legacy

JavaScript

Primarily scripting/programming language for web and beyond

Dynamic type system

Runs in JavaScript engines

.js source files

Prototype-based object model, with class syntax

Common in web frontend and also backend/server environments

1.5 Features of JavaScript

JavaScript provides many important features.

1. Easy to learn

Its basic syntax is relatively easy for students familiar with C, C++ or Java.

Example:

```
let a = 10;
```

```
let b = 20;
```

```
let sum = a + b;
```

```
console.log(sum);
```

2. Lightweight

JavaScript was designed to work efficiently in web browsers.

3. Interpreted / JIT-compiled

Traditionally JavaScript was described as an interpreted language.

Modern JavaScript engines use techniques such as **Just-In-Time (JIT) compilation** to improve performance.

JavaScript is executed by a JavaScript engine, which may interpret and JIT-compile the code during execution.

4. Dynamically typed

The type of a variable can change during execution.

Example:

```
let value = 100;
```

```
console.log(value);
```

```
value = "Hello";
```

```
console.log(value);
```

Initially o/p:

100

is stored.

Later:

Hello

is stored.

5. Object-oriented

JavaScript supports object-oriented programming concepts.

Example:

```
let student = {  
  name: "Rahul",  
  age: 22  
};  
  
console.log(student.name);
```

JavaScript's object model is prototype-based, while modern JavaScript also provides class syntax.

6. Event-driven

JavaScript can respond to user actions such as:

- Mouse clicks
- Keyboard input
- Form submission
- Mouse movement
- Page loading

Example:

```
<button onclick="showMessage()">  
  Click  
</button>  
  
<script>  
  
function showMessage() {
```

```
    alert("Button clicked!");  
}  
</script>
```

7. Case-sensitive

JavaScript is case-sensitive.

These are different:

name

Name

NAME

Example:

```
let name = "Amit";  
console.log(name);
```

But:

```
console.log (Name);
```

will not refer to the same variable.

8. Cross-platform

JavaScript can run on many platforms through JavaScript engines and runtimes.

It is commonly used in:

- Web browsers
 - Servers
 - Desktop applications
 - Mobile applications
 - Development tools
-

1.6 Where is JavaScript Used?

JavaScript is no longer limited to simple browser scripting.

1. Frontend Web Development

Examples:

- Interactive websites
- Menus
- Sliders
- Forms
- Dashboards
- Shopping websites

2. Backend Development

JavaScript can also be used on servers through environments such as Node.js.

Example:

Browser



JavaScript Frontend



Server



Database

3. Web Applications

Examples:

- Gmail-like applications
- Online shopping systems

- Banking interfaces
- Learning management systems

4. Mobile Applications

JavaScript-based frameworks can be used to build mobile applications.

5. Desktop Applications

JavaScript technologies can also be used to create desktop applications.

1.7 JavaScript Engine

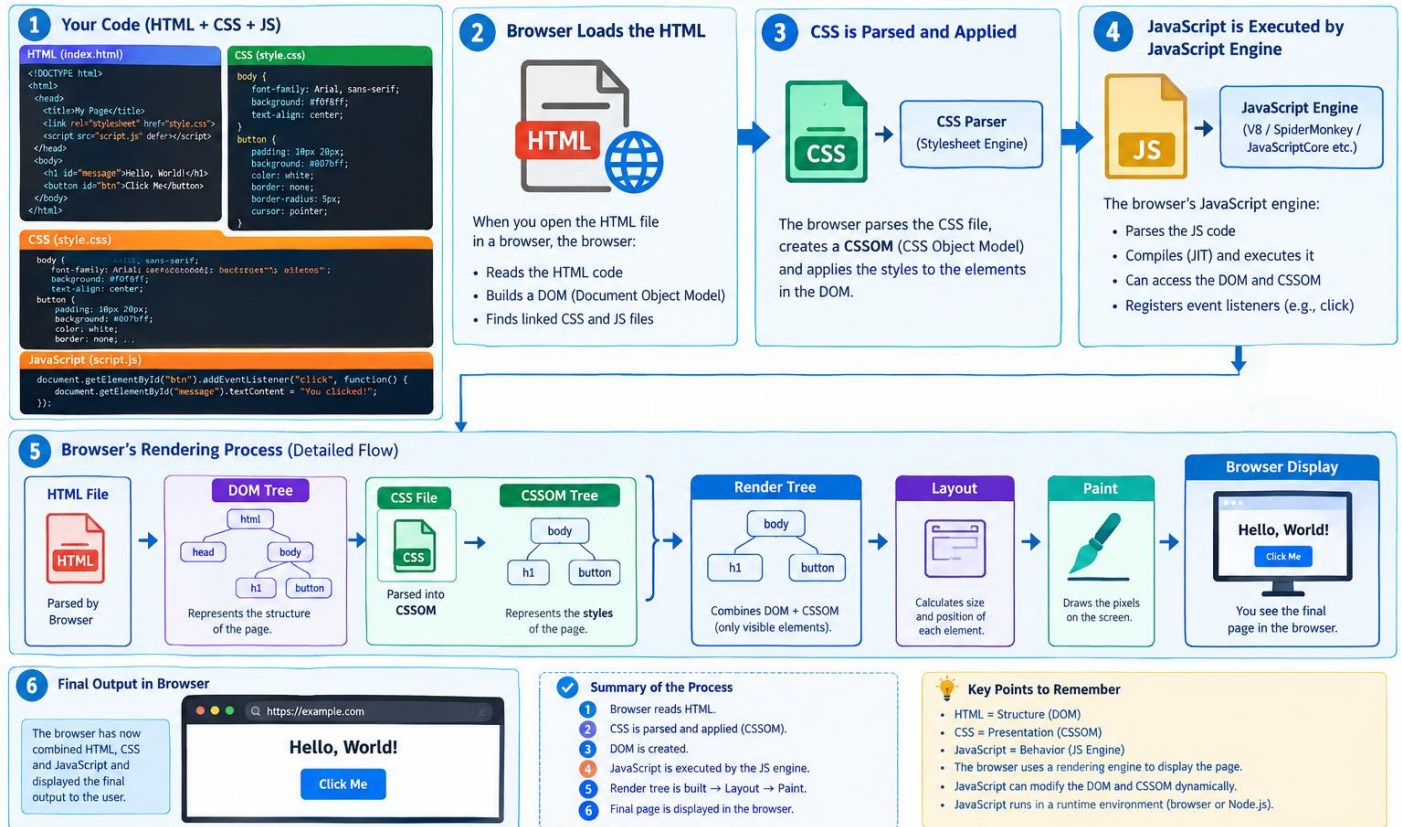
This is an important MCA-level concept.

A **JavaScript engine** is software that executes JavaScript code.

Examples:

Engine	Associated with
V8	Chrome, Node.js
SpiderMonkey	Firefox
JavaScriptCore	Safari
Chakra	Older Microsoft browser ecosystem

How HTML, CSS and JavaScript Code Runs in a Browser (Complete Process – From Code to Output)



Basic execution flow

JavaScript Code



JavaScript Engine



Parsing



Execution / JIT Compilation



Result

1.8 JavaScript Runtime

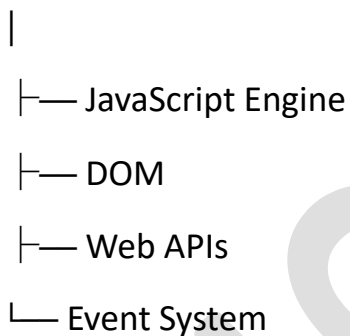
A JavaScript engine alone is not the entire environment.

For example, a browser provides:

- JavaScript engine
- DOM APIs
- Browser APIs
- Event system
- Web APIs

Therefore:

Browser



This distinction becomes very important later when teaching:

- DOM
- Events
- Fetch
- Timers
- Asynchronous JavaScript

1.9 How JavaScript Works in a Browser

Suppose the browser receives:

```
<script>  
  console.log("Hello");
```

`</script>`

The browser processes the HTML and encounters JavaScript.

The JavaScript is then executed by the browser's JavaScript engine.

Conceptually:

HTML File



Browser



JavaScript Engine



Execute JavaScript



Output

1.10 Ways to Add JavaScript to HTML

There are three major ways.

Method 1 — Inline JavaScript

JavaScript can be written directly inside an HTML element's event attribute.

Example:

```
<button onclick="alert('Hello')">
```

```
Click Me</button>
```

Advantage

Very simple for demonstration.

Disadvantage

Not recommended for large applications because HTML and JavaScript become tightly mixed.

1.11 Internal JavaScript

JavaScript can be written inside the <script> element.

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>JavaScript</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Hello World</h1>
```

```
  <script>
```

```
    alert("Welcome to JavaScript");
```

```
  </script>
```

```
</body>
```

```
</html>
```

Here JavaScript is inside:

```
<script>
```

```
</script>
```

1.12 External JavaScript

This is the preferred approach for larger applications.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>JavaScript</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Hello MCA</h1>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

script.js

```
alert ("Welcome to JavaScript!");
```

The HTML file and JavaScript file are separated.

Advantages

- Better organization
- Reusable JavaScript
- Easier maintenance

- Cleaner HTML
- Better for larger projects

1.13 Where Should `<script>` Be Placed?

Students will encounter:

```
<head>
```

and:

```
<body>
```

Example

```
<head>
```

```
<script src="script.js"></script>
```

```
</head>
```

or:

```
<body>
```

```
<script src="script.js"></script>
```

```
</body>
```

A common modern approach is:

```
<script src="script.js" defer></script>
```

inside the `<head>`.

`defer` tells the browser to download the script while parsing HTML and execute it after the document has been parsed.

1.14 First JavaScript Program

Create:

JavaScript/

|

└─ index.html

└─ script.js

index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>First JavaScript Program</title>
```

```
</head>
```

```
<body>
```

```
<h1>My First JavaScript Program</h1>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

script.js

```
console.log("Hello World!");
```

Open the HTML file in a browser.

Open Developer Tools:

F12

Then select:

Console

You should see:

Hello World!

1.15 Output in JavaScript

JavaScript provides several ways to produce output.

1. console.log()

Most commonly used during development and debugging.

```
console.log("Hello World");
```

Output:

Hello World

2. alert()

Displays a browser dialog.

```
alert("Welcome!");
```

3. document.write()

Writes content into the document.

```
document.write("Hello World");
```

Important

document.write() is mainly useful for simple demonstrations and is generally avoided in modern application development.

4. DOM manipulation

Example:

```
<h1 id="heading"></h1>
```

```
<script>
```

```
document.getElementById("heading").textContent =  
    "Hello MCA Students";
```

```
</script>
```

1.16 Comments in JavaScript

Comments are ignored during execution.

Single-line comment

```
// This is a comment
```

```
let age = 22;
```

Multi-line comment

```
/*
```

```
This is a
```

```
multi-line comment
```

```
*/
```

```
let age = 22;
```

Comments are useful for:

- Documentation
- Explanation
- Debugging
- Code maintenance

1.17 JavaScript Statements

A JavaScript program consists of statements.

Example:

```
let a = 10;
```

```
let b = 20;
```

```
let sum = a + b;
```

```
console.log(sum);
```

Here there are multiple statements.

Semicolons are often used to terminate statements:

```
let x = 10;
```

JavaScript has **automatic semicolon insertion** .

1.18 Practical Class

Practical 1

Display:

Hello MCA Students

using console.log().

Practical 2

Display:

Welcome to JavaScript
using alert().

Practical 3

Display:

JavaScript is easy to learn
using document.write().

Practical 4

Create an HTML page containing:

Name

Course

College

and display them using JavaScript.

Practical 5

Create an external JavaScript file.

Structure:

project/

|

├— index.html

└— script.js

Display:

Welcome to MCA JavaScript Programming

using console.log();

Practical 6 — Student Introduction

```
let name = "Rahul";
```

```
let course = "MCA";
```

```
let college = "ABC University";
```

```
console.log("Name:", name);
```

```
console.log("Course:", course);
```

```
console.log("College:", college);
```

1.19 Theory Questions

Very Short Questions

1. What is JavaScript?
2. Who created JavaScript?
3. Is JavaScript the same as Java?
4. What is a JavaScript engine?
5. What is V8?
6. What is an event?
7. What is an external JavaScript file?
8. What is the use of <script>?
9. What is console.log()?
10. Is JavaScript case-sensitive?

Short Questions

1. Explain JavaScript and its major features.
2. Differentiate HTML, CSS and JavaScript.
3. Explain inline, internal and external JavaScript.
4. What is a JavaScript engine?
5. Explain the role of JavaScript in web development.
6. Explain the difference between Java and JavaScript.
7. Explain JavaScript comments.
8. Explain different ways of displaying output in JavaScript.

Long Questions

1. Explain JavaScript in detail, including its history, features and applications.
2. Explain how JavaScript is executed in a web browser.
3. Explain the difference between inline, internal and external JavaScript with examples.
4. Explain JavaScript engines and runtime environments.
5. Explain the relationship between HTML, CSS and JavaScript with suitable examples.